

# ECE 622: Modeling & Verification of Embedded Systems

## Spring 2026: Lab 2: Verification of Timed Automata using UPPAAL

Prof. Daniel Holcomb

---

Submitted by: Shu-Lin Shih and Jaideep Khare  
Student ID: 35199856 and 35193853  
Lab No.: 2  
Submission Date: March 2026

---

## Overview

This report presents model construction, simulation, and symbolic verification for Lab 2 using UPPAAL. Part A analyzes timed automata for the Viking bridge-crossing problem under increasingly complex scenarios. Part B develops and verifies a timed traffic-light and pedestrian-control system, including safety analysis and a robustness extension required for ECE 622.

The workflow used in all sections was: (1) encode model changes in UPPAAL templates/system declarations, (2) state explicit verification queries, (3) run verifier checks, and (4) inspect symbolic or concrete traces to extract schedules and explain results.

## 1 Part A: Bridge-Crossing Timed Automata

### A.1: Four Vikings within 60 minutes

*Determine whether all four Vikings with crossing times 5 min, 10 min, 20 min, and 25 min can reach the safe side within 60 min, and provide a valid schedule.*

Model checking query:

```
E<> Vik_1.safe && Vik_2.safe && Vik_3.safe && Vik_4.safe && time <= 60
```

The query is satisfiable, so a feasible execution exists.

Witness schedule from trace:

| # | Trip (Vikings by index; times in minutes)                      | Start time (min) | Trip origin |
|---|----------------------------------------------------------------|------------------|-------------|
| 1 | Vik <sub>1</sub> + Vik <sub>2</sub> cross to safe side (5+10)  | 0                | Unsafe side |
| 2 | Vik <sub>1</sub> returns with torch (5)                        | 10               | Safe side   |
| 3 | Vik <sub>3</sub> + Vik <sub>4</sub> cross to safe side (20+25) | 15               | Unsafe side |
| 4 | Vik <sub>2</sub> returns with torch (10)                       | 40               | Safe side   |
| 5 | Vik <sub>1</sub> + Vik <sub>2</sub> cross to safe side (5+10)  | 50               | Unsafe side |

Table 1: A.1 crossing schedule (total completion time: 60 min).

The schedule completes all crossings at minute 60, satisfying the time bound exactly.

E> Viking1.safe and Viking2.safe and Viking3.safe and Viking4.safe and time <=60

Figure 1: A.1: UPPAAL verifier for bounded reachability within 60 min (displayed at  $1.5\times$  prior proof scale, width  $\leq$  text width).

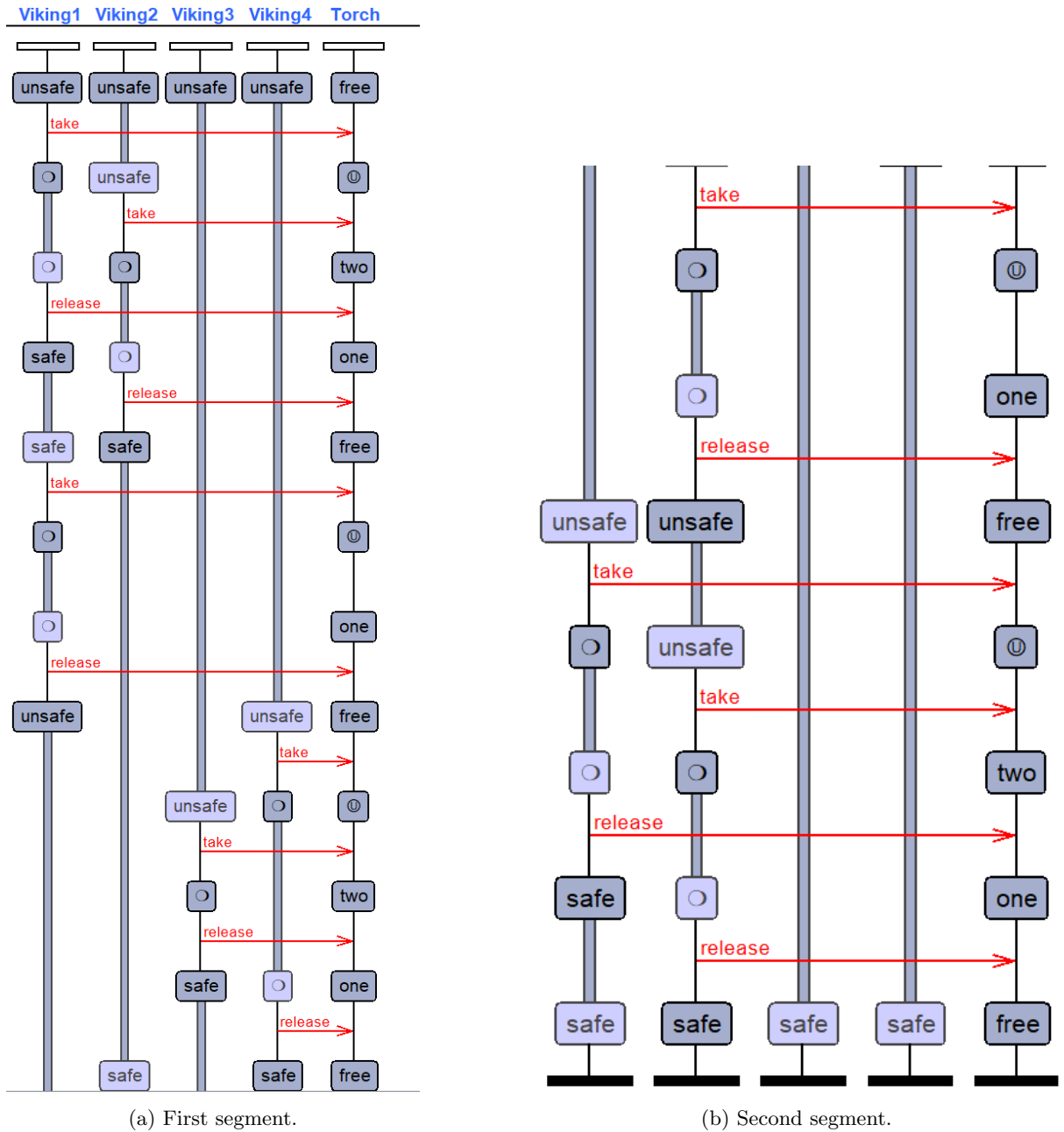


Figure 2: A.1: UPPAAL verifier/trace views for bounded reachability within 60 min (two panels).

## A.2: Add a fifth Viking (10 minutes)

A fifth Viking is added with a 10 min bridge delay (in addition to the existing 10 min Viking). What is the shortest time for all five Vikings to reach the safe side? Describe model changes and queries used.

The fifth Viking is added as a separate template instance with delay `slowest2=10`, composed with the existing four Vikings and the torch:

```
const int slowest2 = 10;
Viking5 = Soldier(slowest2);
system Viking1, Viking2, Viking3, Viking4, Viking5, Torch;
```

Core reachability query (bounded completion time  $T$ ):

```
E<> Vik_1.safe && Vik_2.safe && Vik_3.safe && Vik_4.safe && Vik_5.safe && time <= T
```

A decreasing search over  $T$  and simulator tracing support the schedule below. In the trace,  $Vik_1$  and  $Vik_2$  cross together and  $Vik_1$  returns with the torch; later  $Vik_1$  and  $Vik_2$  cross again with  $Vik_1$  returning; then

Vik<sub>1</sub> crosses with Vik<sub>3</sub> and Vik<sub>1</sub> returns; finally Vik<sub>1</sub> and Vik<sub>4</sub> cross together. The shortest completion time obtained for this model is 80 min.

| # | Trip (Vikings by index; times in minutes)        | Start time (min) | Trip origin |
|---|--------------------------------------------------|------------------|-------------|
| 1 | Vik <sub>1</sub> + Vik <sub>2</sub> cross (5+10) | 0                | Unsafe side |
| 2 | Vik <sub>1</sub> returns with torch (5)          | 10               | Safe side   |
| 3 | Vik <sub>1</sub> + Vik <sub>2</sub> cross (5+10) | 15               | Unsafe side |
| 4 | Vik <sub>1</sub> returns with torch (5)          | 25               | Safe side   |
| 5 | Vik <sub>1</sub> + Vik <sub>3</sub> cross (5+20) | 30               | Unsafe side |
| 6 | Vik <sub>1</sub> returns with torch (5)          | 50               | Safe side   |
| 7 | Vik <sub>1</sub> + Vik <sub>4</sub> cross (5+25) | 55               | Unsafe side |

Table 2: A.2 crossing schedule (witness trace; total completion 80 min).

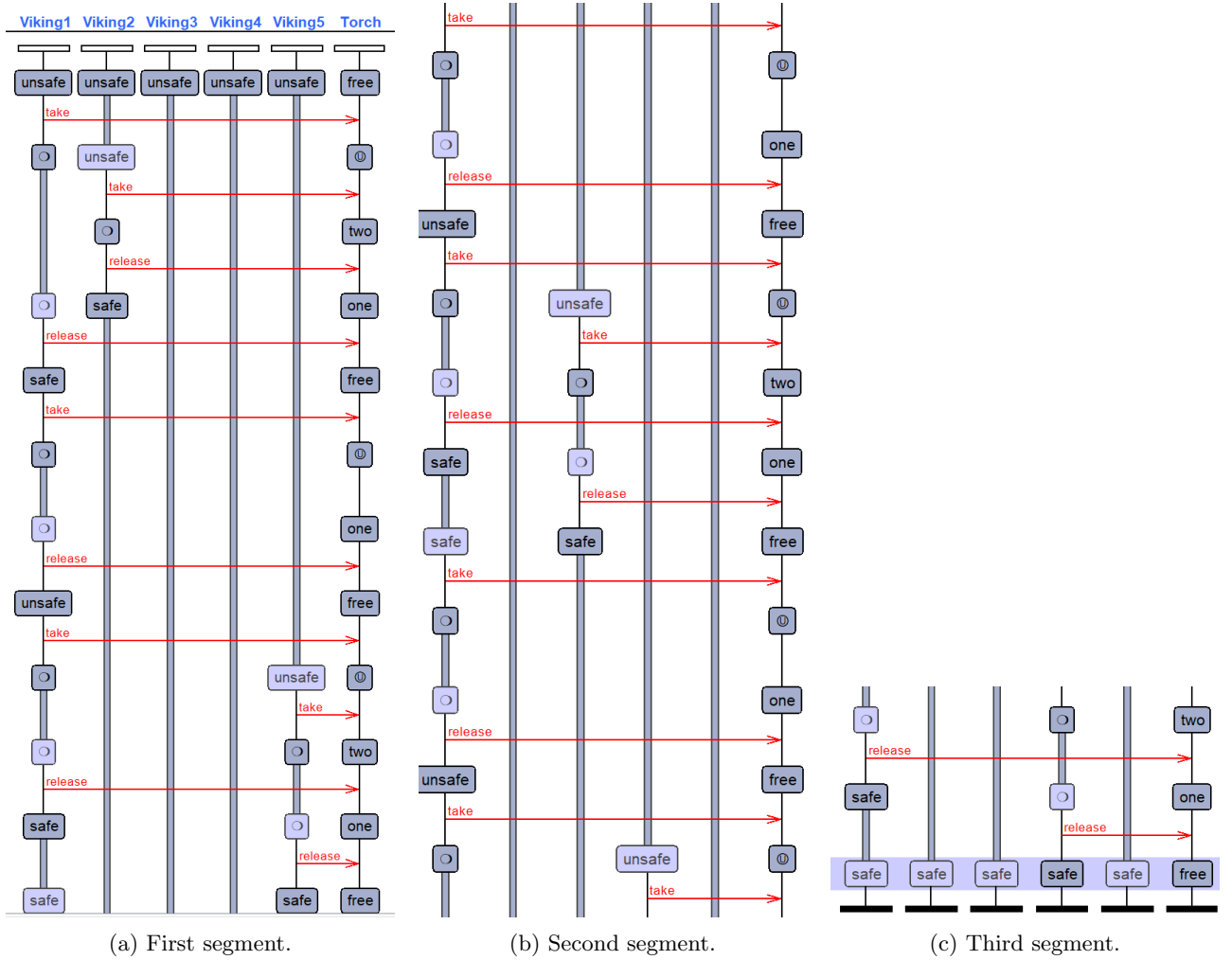


Figure 3: A.2: UPPAAL simulator trace split across three views.

### A.3: Bridge and valley scenario

Building on A.2, each Viking may use a longer valley path (no torch; valley delay is  $4 \times$  each Viking's bridge delay). The bridge still carries two Vikings; the valley has unlimited capacity. What is the shortest time for all five to reach the safe side? Include a schedule with a column indicating bridge vs. valley.

The soldier template gains a valley location with clock  $y$  and invariant  $y \leq 4 \cdot \text{delay}$ , matching the lab requirement that the valley takes four times as long as the bridge for each Viking. Verification uses bounded reachability, e.g.

```
E<> Vik_1.safe && Vik_2.safe && Vik_3.safe && Vik_4.safe && Vik_5.safe && time <= 40
```

The shortest time for all five to reach the safe side in this model is 40 min. In the symbolic trace used for the table, Vik<sub>3</sub> and Vik<sub>4</sub> cross the bridge together while Vik<sub>1</sub>, Vik<sub>2</sub>, and Vik<sub>5</sub> (the two 10 min Vikings) take the valley concurrently at  $t = 0$ .

| # | Trip (Vikings by index; times in minutes)         | Start time (min) | Trip origin | Bridge or valley |
|---|---------------------------------------------------|------------------|-------------|------------------|
| 1 | Vik <sub>3</sub> + Vik <sub>4</sub> cross (20+25) | 0                | Unsafe side | bridge           |
| 2 | Vik <sub>1</sub> crosses (5)                      | 0                | Unsafe side | valley           |
| 3 | Vik <sub>2</sub> crosses (10)                     | 0                | Unsafe side | valley           |
| 4 | Vik <sub>5</sub> crosses (10)                     | 0                | Unsafe side | valley           |

Table 3: A.3 schedule excerpt (parallel starts at  $t = 0$  where applicable).

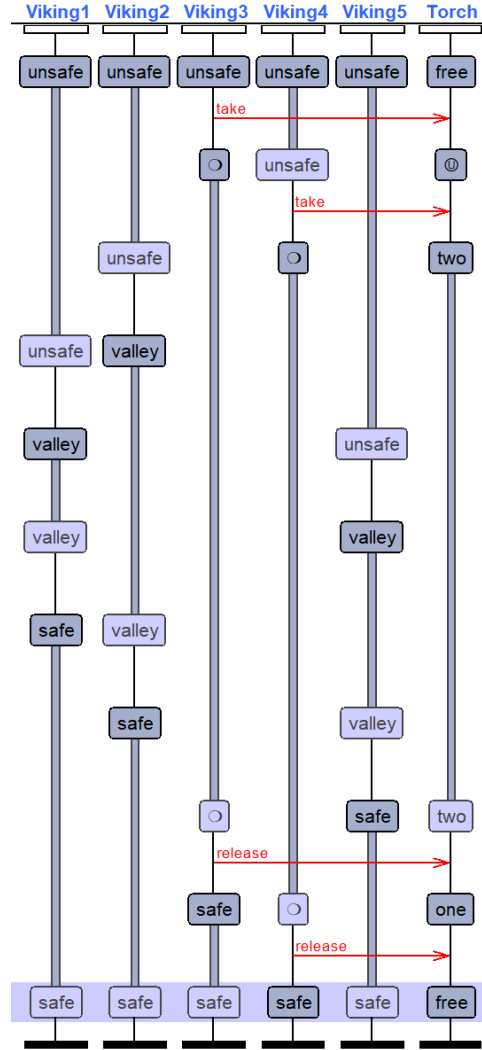


Figure 4: A.3: UPPAAL simulation/symbolic trace for the bridge-valley model.

## 2 Part B: Traffic-Light and Crosswalk Verification

### B.1: Discrete-model reasoning (before timed implementation)

For each property, a reachability-oriented check was defined conceptually and then evaluated by model intuition over the discrete behavior (no tool reachability on the discrete model in this step).

| Prop. | Reachability check idea                                                                                    | Expected truth   | Rationale (intuition)                                                                                                                                                                                                    |
|-------|------------------------------------------------------------------------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (a)   | It is impossible for the traffic light to be in the pending state 70s after the initial condition.         | False            | The claim of impossibility is false: pending at <code>globalTime==70</code> can occur if a pedestrian presses the button shortly before the end of a green phase, leaving the controller in pending around that horizon. |
| (b)   | It is possible for the traffic and crosswalk lights to be green at the same time.                          | False            | Simultaneous green on both is not expected in normal signaling; the discrete interaction does not support both-green overlap.                                                                                            |
| (c)   | The traffic light will never stay in the green (non-pending) location longer than 120s.                    | True             | Timing guards/invariants cap the green phase at 120s or below.                                                                                                                                                           |
| (d)   | The traffic light will never stay in the pending location for 45s or longer.                               | False            | Pending can persist for up to about 60s in plausible behaviors, so occupying pending for at least 45s is reachable.                                                                                                      |
| (e)   | The traffic light will never stay in the pending location for 65s or longer.                               | True             | The structure of pending timing prevents occupancy at or above 65s.                                                                                                                                                      |
| (f)   | It is possible that the crosswalk is red while the traffic light is green.                                 | True             | This phase is part of normal operation (e.g., before pedestrian service).                                                                                                                                                |
| (g)   | Whenever the crosswalk is red, the traffic light must be green.                                            | False            | The model does not enforce this implication; all-red/yellow transitional states break it.                                                                                                                                |
| (h)   | The 3rd occurrence of <code>sigG</code> must not happen within the first 300s after the initial condition. | False (violated) | A third <code>sigG</code> can occur before 300s under fast cycling.                                                                                                                                                      |
| (i)   | The 3rd occurrence of <code>sigG</code> must not happen within the first 400s after the initial condition. | False (violated) | A third <code>sigG</code> can still occur within 400s; the property is violated similarly to (h).                                                                                                                        |

Table 4: B.1 qualitative checks and expected outcomes (intuition only).

## B.2: Timed automata model construction

The timed UPPAAL model includes:

- Traffic-light and crosswalk-light automata connected by broadcast/synchronization channels (e.g., `sigR`, `sigG`, `sigY`, `pedG`, `pedR`).
- Non-deterministic pedestrian input trigger (`pedinput`): implemented as a self-looping transmitter so that the pedestrian button channel can fire concurrently with other synchronizations. The self-loop does not encode meaningful defaulting behavior; it exists so that UPPAAL’s channel semantics allow the input to be offered in the same step as other transitions when needed.
- Receiver/sink automata for outputs that would otherwise lack a listener: unused outputs are connected to simple self-looping receivers that absorb signals, which keeps broadcast sends from blocking the rest of the model.
- Urgent intermediate locations where needed to split transitions that conceptually combine input and output events.

The model was exercised with concrete simulation and simple reachability checks before formal verification in B.3.

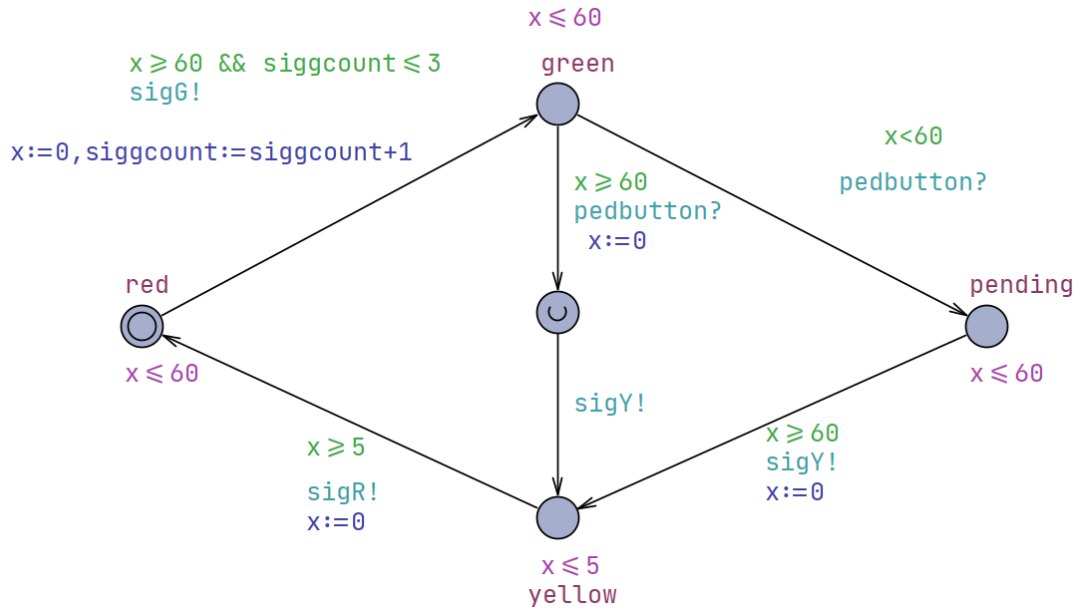


Figure 5: Timed automata for the traffic-light system (B.2).

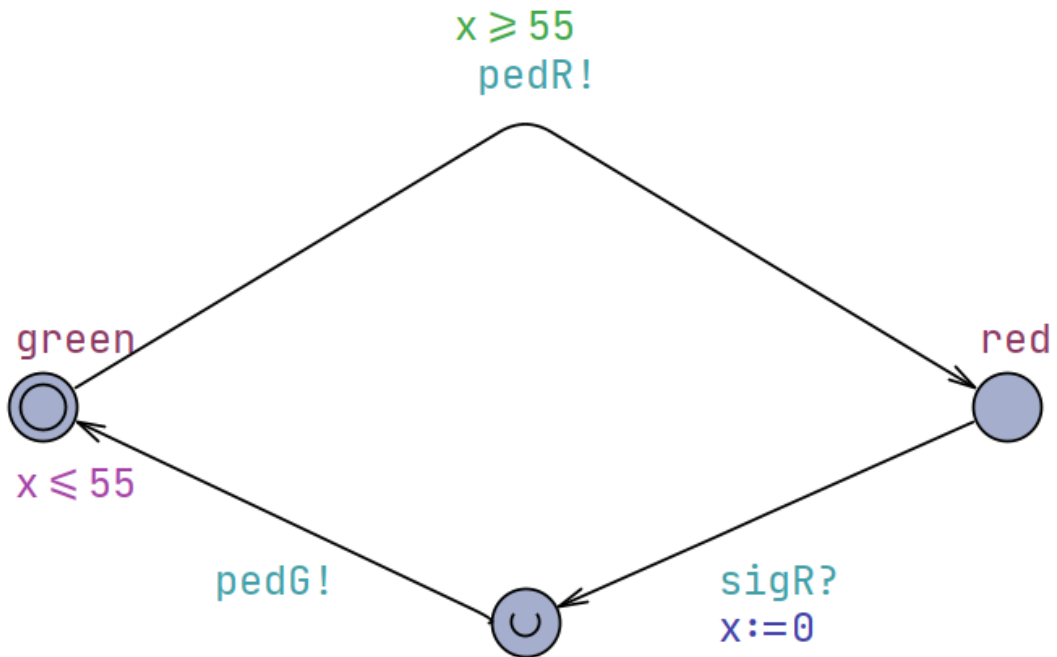


Figure 6: Crosswalk / pedestrian-side signal system (B.2).

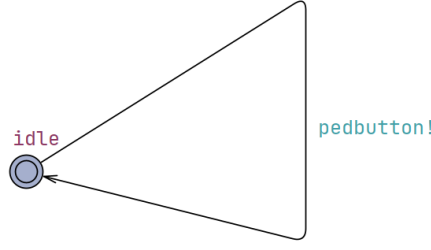


Figure 7: Non-deterministic `pedinput` trigger (self-loop for channel firing). Image width is half of the first two B.2 figures.

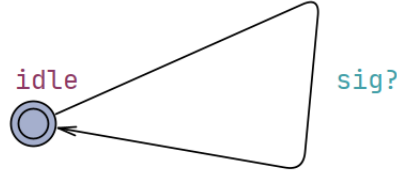


Figure 8: Receiver/sink automata for unused outputs. Image width is half of the first two B.2 figures.

```

E<> crosswalklight1.green
E<> crosswalklight1.red
E<> trafficlightcontroller1.pending
E<> trafficlightcontroller1.green
E<> trafficlightcontroller1.yellow
A[] not deadlock

```



Figure 9: Reachability / sanity checks used while testing the B.2 model.

### B.3: Query mapping and verification outcomes

Each B.1 English statement was translated into a UPPAAL query over the B.2 timed model. The verifier panel confirms results for all checks in one model snapshot.

| Prop. | Representative UPPAAL query form                                   | UPPAAL outcome | Satisfied (true/false) |
|-------|--------------------------------------------------------------------|----------------|------------------------|
| (a)   | <code>E&lt;&gt; (globalTime==70 &amp;&amp; traffic.pending)</code> | Reachable      | true                   |
| (b)   | <code>E&lt;&gt; (traffic.green &amp;&amp; crosswalk.green)</code>  | Reachable      | true                   |
| (c)   | <code>A[] (traffic.green imply tGreen&lt;=120)</code>              | Satisfied      | true                   |
| (d)   | <code>A[] not (traffic.pending &amp;&amp; tPend&gt;=45)</code>     | Violated       | false                  |
| (e)   | <code>A[] not (traffic.pending &amp;&amp; tPend&gt;=65)</code>     | Satisfied      | true                   |
| (f)   | <code>E&lt;&gt; (traffic.green &amp;&amp; crosswalk.red)</code>    | Reachable      | true                   |
| (g)   | <code>A[] (crosswalk.red imply traffic.green)</code>               | Violated       | false                  |
| (h)   | <code>A[] (sigGCount&lt;3 or globalTime&gt;300)</code>             | Violated       | false                  |
| (i)   | <code>A[] (sigGCount&lt;3 or globalTime&gt;400)</code>             | Violated       | false                  |

Table 5: B.3 query mapping, UPPAAL outcome, and Boolean satisfaction of the checked formula (true if the query is satisfied in UPPAAL).

*Note.* Property (b) is marked reachable here; if the discrete intuition in Table 4 suggested otherwise, the timed B.2 model should be treated as authoritative for B.3.

```

//i
E> siggcount ==3 and globaltime ≤ 400
//h
E> siggcount ==3 and globaltime ≤ 300
//g
E> trafficlightcontroller1.yellow and crosswalklight1.red
E> trafficlightcontroller1.red and crosswalklight1.red
//f
E> trafficlightcontroller1.green and crosswalklight1.red
//e
E> trafficlightcontroller1.pending and trafficlightcontroller1.x >65
//d
E> trafficlightcontroller1.pending and trafficlightcontroller1.x >45
//c
E> trafficlightcontroller1.green and trafficlightcontroller1.x >90
//b
E> trafficlightcontroller1.green and crosswalklight1.green
//a
E> trafficlightcontroller1.pending and globaltime = 70

```



Figure 10: Verifier window: B.3 queries with pass/fail indicators.

#### B.4: Pedestrian component and unsafe crossing-time range

The previous button-generator and `pedG` receiver were replaced with an explicit pedestrian automaton having three locations: *Idle*, *Waiting*, and *Crossing*. Parameter `pedCrossTime` controls required crossing time.

Bad condition: the system is unsafe when traffic becomes green while the pedestrian remains in the crossing location.

Observed threshold: reachability checks show the bad condition is unreachable at `pedCrossTime=59` but reachable at `pedCrossTime=60`. The smallest unsafe value is therefore 60, and values  $\geq 60$  are unsafe for this model.

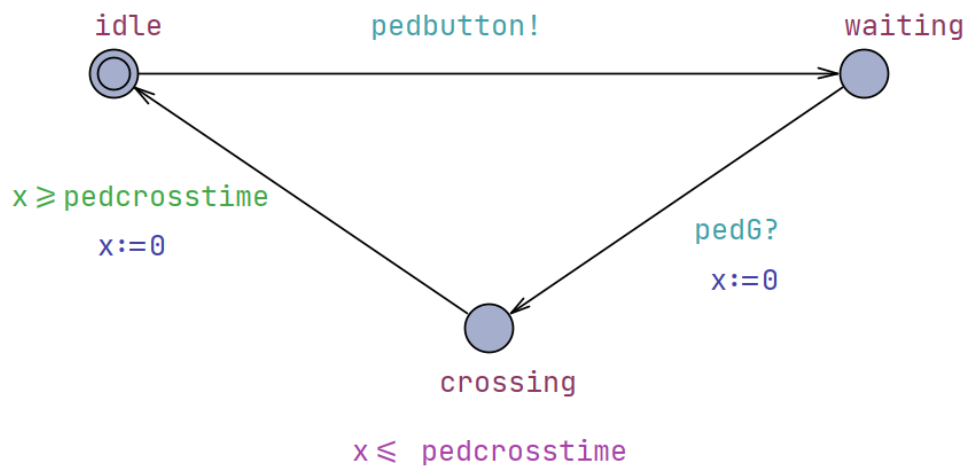


Figure 11: Pedestrian template with `pedCrossTime` and crossing-state timing (B.4).

```

// receiver pedG = receiver (ped
pedstrain1=pedstrain(59);

E> pedstrain1.crossing and trafficlightcontroller1.green

```



Figure 12: B.4 safe case at `pedCrossTime=59`: query/clause (top) and trace (bottom).

```
pedestrian1=pedestrian(60);
```

```
E> pedestrian1.crossing and trafficlightcontroller1.green
```

Figure 13: B.4 unsafe case at `pedCrossTime=60`: query/clause (top) and trace (bottom).

## B.5 (ECE 622): Robustness change for arbitrary crossing time

Lab 2 asks for a more robust model such that safety holds regardless of how long the pedestrian takes to cross; explain the change; repeat the B.4 safety query at a `pedCrossTime` value that was unsafe before and show it is safe after the change; submit `_b5.xml` with the report.

To remove dependence on a bounded pedestrian crossing duration, a synchronization signal `done` was introduced from the pedestrian automaton to the traffic controller. The controller defers returning to green until the pedestrian reports crossing completion.

The previously unsafe test point from B.4 (`pedCrossTime=60`) becomes safe after adding the `done` handshake and updating controller guards accordingly.

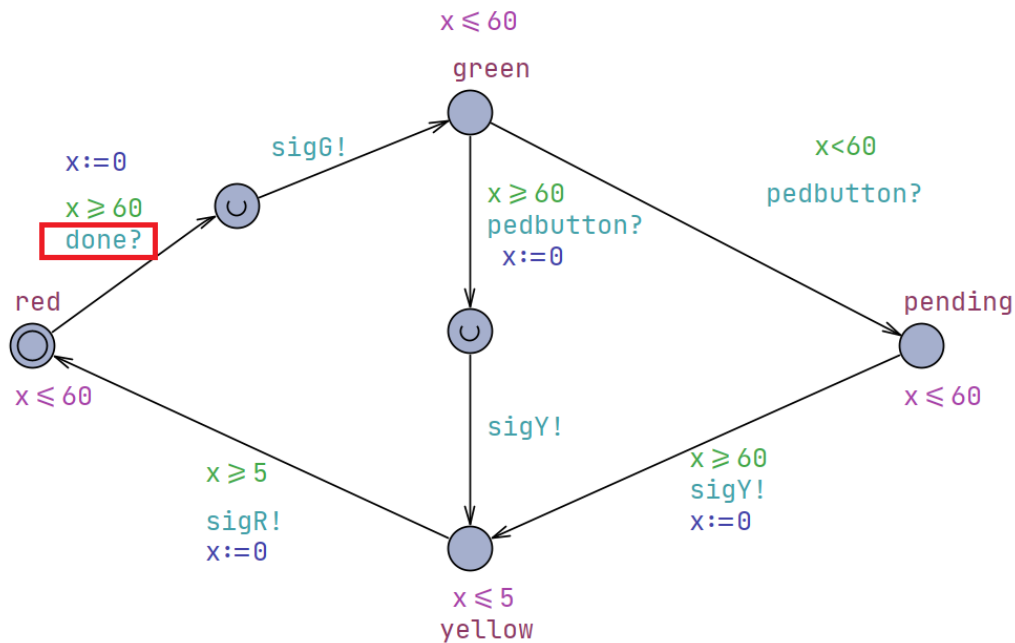


Figure 14: Traffic controller with `done` handshake (B.5).

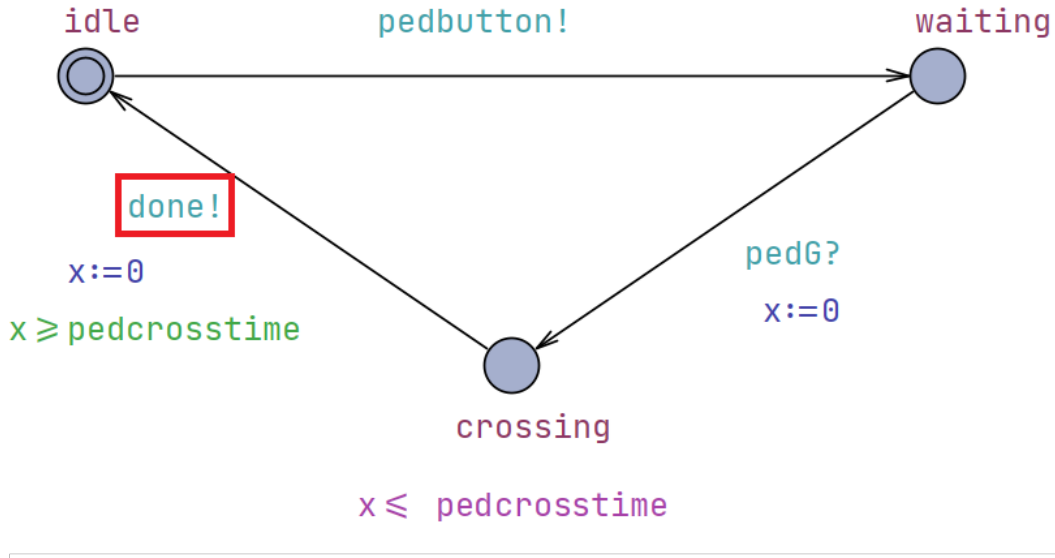


Figure 15: Pedestrian model cooperating with **done** (B.5).

```
E> pedestrain1.crossing and trafficlightcontroller1.green
```

Figure 16: B.5: safety re-check at `pedCrossTime=60` (now safe).

**pedestrain1=pedestrain(60);**

Figure 17: B.5: corresponding query/clause for the unsafe condition at `pedCrossTime=60`.

```
E> Viking1.safe and Viking2.safe and Viking3.safe and Viking4.safe and time ≤ 60
```

Figure 18: B.5: additional confidence checks (e.g. crossing allowed under robust model).

| # | Lab 2 B.5 requirement (paraphrase)                                                                 | What this report demonstrates                                                                   | OK? |
|---|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-----|
| 1 | Robust model: safe regardless of crossing duration.                                                | <b>done</b> handshake; green deferred until crossing completes (Figs. 14–15).                   | Y   |
| 2 | Explain the change in the report.                                                                  | Text and figures; <b>done</b> removes reliance on a fixed <code>pedCrossTime</code> for safety. | Y   |
| 3 | Repeat B.4 safety query at a previously unsafe <code>pedCrossTime</code> ; show safe after change. | Figs. 16–17: bad condition unreachable at <code>pedCrossTime=60</code> .                        | Y   |
| 4 | Simulate / use extra queries for confidence.                                                       | Fig. 18 and B.2-style checks.                                                                   | Y   |
| 5 | Submit <code>_b5.xml</code> with the report.                                                       | Course-style filename (e.g. <code>*_b5.xml</code> ) included with submission materials.         | Y   |

Table 6: B.5 alignment checklist: lab wording vs. evidence in this report (for grading).

## Conclusion

This lab demonstrates how UPPAAL supports rigorous analysis of both scheduling and reactive control problems. In Part A, bounded reachability and trace extraction were used to derive crossing schedules under increasingly rich constraints. In Part B, timed automata and formal queries were used to relate informal predictions to tool results, to expose unsafe regimes, and to validate a robustness refinement that removes dependence on a fixed pedestrian timing assumption.